■ NODE.JS ■

Complete Learning & Interview Handbook

Part 1: Fundamentals to Intermediate

11 In-Depth Chapters	**100 Interview Q&As**	**50+ Code Examples**
Event Loop Deep Dive	**Async Patterns**	**Best Practices**
Cheat Sheets	**Quick Revision**	**Common Traps**

Covers: Fundamentals · Architecture · Runtime · Modules · NPM · Async · Errors · Events

Generated: June 2026

TABLE OF CONTENTS

- Internal Working

Chapter 10: Interview Questions Bank

- 50 Beginner Questions
- 50 Intermediate Questions

Chapter 11: Revision Notes

- Cheat Sheets
- Quick Revision Tables
- Common Traps
- FAQ

CHAPTER 1

Introduction to Node.js

1.1 What is Node.js?

DEFINITION

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 JavaScript engine. It allows developers to execute JavaScript code on the server side, outside a web browser. Node.js provides an event-driven, non-blocking I/O model that makes it lightweight and efficient, perfectly suited for data-intensive real-time applications that run across distributed devices.

1.2 Why Was Node.js Created?

WHY IT EXISTS

Before Node.js, server-side development required languages like PHP, Python, Ruby, or Java. Ryan Dahl created Node.js in 2009 because he was frustrated with the limitations of Apache HTTP Server — specifically its blocking I/O model. Traditional servers spawn a new thread for every incoming connection, consuming RAM and CPU. Dahl's insight was: JavaScript's single-threaded, event-loop model already handles async operations perfectly in browsers — why not bring that to the server?

- Eliminate thread-per-request overhead
- Enable high concurrency with low resource usage
- Allow developers to use one language (JavaScript) on both front-end and back-end
- Build real-time apps like chats, live dashboards, streaming APIs

1.3 History of Node.js

Year	Event
2009	Ryan Dahl presents Node.js at JSConf EU — initial release
2010	npm (Node Package Manager) created by Isaac Schlueter
2011	Node.js v0.4 — major Windows support added
2014	io.js fork created due to governance disputes
2015	io.js and Node.js merge under the Node.js Foundation — Node v4 (LTS)
2018	Node.js Foundation merges with JS Foundation → OpenJS Foundation
2020	Node.js v14 LTS — V8 8.0, optional chaining, nullish coalescing
2022	Node.js v18 LTS — native fetch API, test runner built-in
2023-24	Node.js v20/v22 — permission model, ESM improvements, native WebSocket

1.4 The V8 Engine

INTERNAL WORKING

V8 is Google's open-source, high-performance JavaScript and WebAssembly engine written in C++. It is used in Google Chrome and Node.js. V8 compiles JavaScript directly to native machine code before executing it, rather than interpreting bytecode.

V8 Compilation Pipeline:

1. Parser → Converts JS source to Abstract Syntax Tree (AST)
2. Ignition (Interpreter) → Compiles AST to bytecode
3. TurboFan (Optimizing Compiler) → JIT-compiles hot functions to optimized machine code
4. Orinoco (Garbage Collector) → Manages memory with generational GC

```
// V8 processes this JS:
function add(a, b) { return a + b; }
add(1, 2); // V8 detects 'hot' function
// TurboFan compiles it to optimized machine code
// Subsequent calls skip interpretation entirely
```

1.5 JavaScript Runtime

A JavaScript runtime is an environment that provides everything needed to run JS code. Node.js runtime includes:

Component	Role	Provider
V8 Engine	Execute JS / Compile to machine code	Google
libuv	Event loop, async I/O, thread pool	C library
Node.js Bindings	Bridge between JS and C++ system calls	Node core
Node Core Modules	fs, http, path, crypto, etc.	Node standard library
npm	Package ecosystem	npm Inc.

1.6 Browser JS vs Node.js

Feature	Browser JS	Node.js
Environment	Web browser	Server / Terminal
Global Object	window	global / globalThis
DOM Access	Yes (document, window)	No DOM
File System	Restricted (sandboxed)	Full access via fs module
Modules	ES Modules (type=module)	CommonJS + ES Modules
Networking	XMLHttpRequest / fetch	http, https, net modules
Threading	Single thread + Web Workers	Single thread + Worker Threads
Process Control	No	process.exit(), process.env

Feature	Browser JS	Node.js
Purpose	UI interaction	Server-side logic, APIs, tooling

1.7 Advantages of Node.js

High Performance: V8 + async I/O handles tens of thousands of concurrent connections

Single Language: Use JavaScript everywhere — front-end, back-end, CLI, scripts

Rich Ecosystem: npm has 2M+ packages for every imaginable use case

Real-Time: WebSocket / SSE support built-in — great for chat, gaming, live data

Microservices: Lightweight footprint suits containerised microservice architectures

Active Community: Massive community, regular LTS releases, well-funded foundation

1.8 Limitations of Node.js

CPU-Intensive Tasks: Single thread — heavy computation blocks the event loop

Callback Complexity: Historical: callback hell (mitigated by Promises / async-await)

Immaturity in Some Areas: Some enterprise patterns (e.g. complex ORM) better in Java/.NET

Error Handling: Uncaught exceptions crash the process unless handled explicitly

Relational DB Support: Historically weaker than Java/Python ORMs (Prisma improving this)

1.9 When to USE Node.js

■ Best Use Cases

- REST / GraphQL APIs with high concurrency
- Real-time apps: chat, notifications, live collaboration
- Microservices and serverless functions (AWS Lambda)
- CLI tools and build tooling (webpack, eslint, etc.)
- Streaming applications: video/audio processing pipelines
- BFF (Backend for Frontend) proxy layers

1.10 When NOT to Use Node.js

■ Poor Fit

- CPU-intensive computation: image processing, ML model training, video encoding
- Complex relational data with heavy transactions (consider PostgreSQL + Java/Go)
- Systems programming requiring low-level memory control (use C, Rust)
- Applications needing multi-threaded parallelism by default (use Go, Java)

Chapter 1 — Interview Questions

Q1. What is Node.js?

A: Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model enabling efficient, scalable server-side applications.

Q2. What is the difference between Node.js and a browser's JavaScript runtime?

A: Browsers provide window, DOM, and sandboxed file/network access. Node.js provides global, process, and full OS access (file system, networking, environment variables) but has no DOM.

Q3. What is the V8 engine?

A: V8 is Google's open-source JS engine written in C++. It compiles JS to machine code via JIT compilation (Ignition bytecode + TurboFan optimiser). Used in Chrome and Node.js.

Q4. Who created Node.js and why?

A: Ryan Dahl created it in 2009 frustrated by Apache's blocking thread-per-request model. He leveraged JS's event-loop model to build a non-blocking server runtime.

Q5. What is libuv in Node.js?

A: libuv is a C library that powers Node's event loop, async I/O, DNS resolution, file system operations, and maintains a thread pool for offloading CPU-bound tasks away from the main thread.

CHAPTER SUMMARY

- ✓ *Node.js = V8 Engine + libuv + Node Core Modules + npm ecosystem*
- ✓ *Created in 2009 by Ryan Dahl to solve blocking I/O problems of traditional servers*
- ✓ *Uses single-threaded event loop with non-blocking async I/O for high concurrency*
- ✓ *Ideal for: APIs, real-time apps, microservices, streaming, CLI tools*
- ✓ *Avoid for: CPU-heavy tasks, complex relational workloads, systems programming*
- ✓ *Browser JS has window/DOM; Node.js has process/fs/os — no DOM*

CHAPTER 2

Node.js Architecture

2.1 Single-Threaded Model

Node.js operates on a single main thread for JavaScript execution. Unlike traditional servers (e.g., Apache) that create a new OS thread per request, Node.js uses ONE thread to handle ALL incoming requests. This might sound limiting, but it is extremely efficient because Node's I/O operations are non-blocking — the thread never sits idle waiting for disk or network.

```
// Traditional blocking server (pseudo-code):
// Thread 1: handles Request A → waits for DB → responds
// Thread 2: handles Request B → waits for DB → responds
// Thread 3: handles Request C → waits for DB → responds
// 10,000 requests = 10,000 threads = massive RAM usage

// Node.js non-blocking (pseudo-code):
// 1 Thread: handles Request A → initiates DB query (async)
// handles Request B → initiates DB query (async)
// handles Request C → initiates DB query (async)
// Callbacks fire as results arrive — no waiting!
```

2.2 Event-Driven Architecture

Node.js is fundamentally event-driven. Everything in Node.js is built around events. When an I/O operation completes (file read, HTTP response), it emits an event. Registered callback functions (event listeners) respond to those events. This is the Observer Pattern at the core of Node's design.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Register a listener (observer)
emitter.on('data', (payload) => {
console.log('Received:', payload);
});

// Emit event later (after async operation)
setTimeout(() => {
emitter.emit('data', { userId: 42, name: 'Alice' });
}, 1000);
```

2.3 Non-Blocking I/O — Deep Dive

INTERNAL WORKING

Non-blocking I/O means that when Node.js initiates an I/O operation (file read, network request, database query), it does NOT pause execution. Instead, it registers a callback and continues processing other code. When the I/O completes, the OS signals libuv, which places the callback in the event queue to be executed when the call stack is empty.

How It Works Internally:

Step 1: JS calls fs.readFile() — passes callback function

Step 2: Node bindings send the request to libuv

Step 3: libuv delegates to the OS (epoll on Linux, kqueue on macOS, IOCP on Windows)

Step 4: Main thread continues executing OTHER code

Step 5: OS completes file read, notifies libuv

Step 6: libuv places the callback in the I/O callbacks queue

Step 7: Event loop picks up callback on next iteration and executes it

```
const fs = require('fs');

console.log('1. Start');

fs.readFile('data.txt', 'utf8', (err, data) => {
// This executes AFTER the main thread finishes
console.log('3. File read complete:', data.length, 'bytes');
});

console.log('2. This runs immediately — not blocked!');

// Output:
// 1. Start
// 2. This runs immediately — not blocked!
// 3. File read complete: 1024 bytes
```

2.4 The Event Loop — Overview

The Event Loop is the heart of Node.js. It is implemented in libuv and runs continuously, checking if there are tasks to execute. It allows Node.js to perform non-blocking I/O operations by offloading operations to the OS whenever possible.

Event Loop Phases (libuv):

Phase	Queue	What Happens
1. timers	Timer callbacks	Executes setTimeout / setInterval callbacks whose threshold has elapsed
2. pending callbacks	I/O error callbacks	Executes I/O callbacks deferred to next loop iteration
3. idle / prepare	Internal	Used internally by libuv
4. poll	New I/O events	Retrieve new I/O events; execute I/O-related callbacks. Node blocks here if nothing else pending

Phase	Queue	What Happens
5. check	setImmediate callbacks	setImmediate() callbacks execute here
6. close callbacks	Close events	socket.on('close') and similar cleanup callbacks

Visual: Event Loop Execution Order

```
┌─────────────────────────────────────────────────────────────┐
│ EVENT LOOP │
│ │
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────────┐ │
│ │ timers │──▶│ poll │──▶│ check (setImm.) │ │
│ └──────────────┘ └──────────────┘ └──────────────────┘ │
│ ▲ │ │ │
│ └──────────────────────────────────────────┘ │
│ │
│ Between each phase: process.nextTick + microtasks │
│ (Promises .then) are drained FIRST │
└─────────────────────────────────────────────────────────────┘
```

2.5 How Node.js Handles Thousands of Requests

The key insight: I/O is slow; CPU is fast. A typical DB query takes 5-100ms. During that time, the CPU is idle in a blocking model. Node's event loop keeps the CPU busy processing OTHER requests while waiting for I/O. This achieves massive concurrency with a single thread.

Thread Pool (libuv)

For tasks the OS cannot make truly async (DNS lookups, some file system ops, crypto), libuv uses a thread pool (default: 4 threads, configurable via UV_THREADPOOL_SIZE). These tasks run on background threads while the event loop continues.

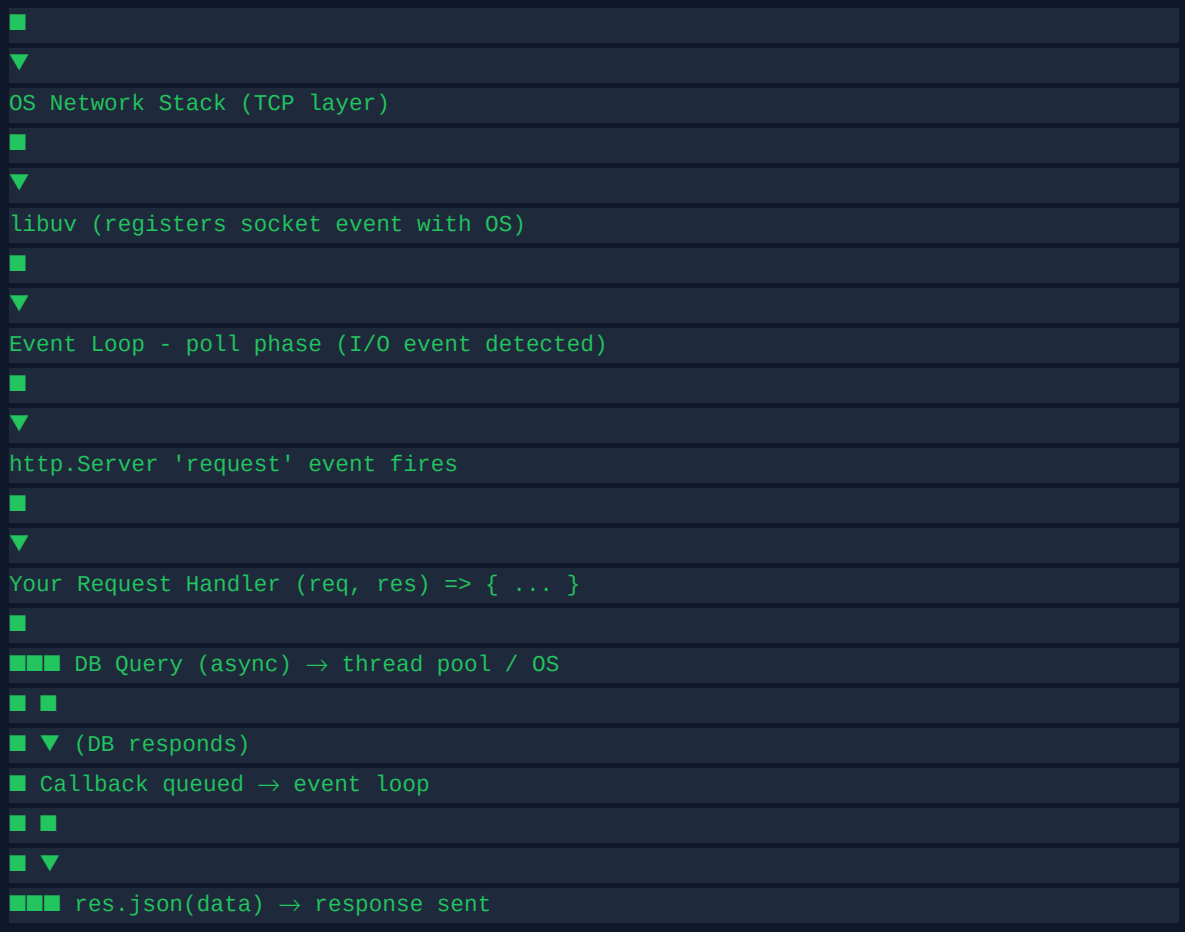
```
// Increase thread pool for CPU-heavy async tasks:
process.env.UV_THREADPOOL_SIZE = 8; // Up to 128

// Tasks that use the thread pool:
// - fs operations (except on Linux with io_uring)
// - crypto (pbkdf2, scrypt, randomBytes)
// - DNS (dns.lookup)
// - zlib (compression)

// Tasks handled directly by OS (no thread pool needed):
// - TCP/UDP sockets
// - Pipes
// - TTY
```

2.6 Request Lifecycle in Node.js (HTTP)

```
Client Request
```



Chapter 2 — Interview Questions

Q1. Explain Node.js single-threaded model.

A: Node.js has one main JS thread for execution. I/O operations are delegated to libuv/OS asynchronously. The main thread never blocks — it registers callbacks and continues. This achieves concurrency without multiple threads.

Q2. What is non-blocking I/O?

A: Non-blocking I/O means initiating an I/O operation and immediately returning control to the caller. The result is delivered via a callback when ready. The thread continues other work instead of waiting.

Q3. What are the phases of the Event Loop?

A: timers → pending callbacks → idle/prepare → poll → check → close callbacks. Between each phase, process.nextTick() and Promise microtasks are drained.

Q4. What is libuv?

A: libuv is a C library providing Node's event loop, async I/O, thread pool, DNS, file system, and networking across platforms (Linux epoll, macOS kqueue, Windows IOCP).

Q5. How does Node handle 10,000 concurrent requests?

A: Node registers each incoming connection as an event. I/O (DB queries, file reads) is delegated asynchronously. The single thread processes callbacks as responses arrive — no threads blocked waiting. This uses far less memory than thread-per-request models.

CHAPTER SUMMARY

✓ *Node.js uses ONE main thread for JS + libuv for async I/O delegation*

✓ *Non-blocking I/O: initiate operation → register callback → continue → callback fires when done*

✓ *Event loop has 6 phases; microtasks (Promises, nextTick) drain between each phase*

✓ *libuv thread pool (4 by default) handles DNS, file ops, crypto off the main thread*

✓ *High concurrency achieved by keeping CPU busy processing other requests during I/O waits*

CHAPTER 3

JavaScript Runtime Concepts

3.1 Execution Context

Every time JavaScript code runs, it runs inside an Execution Context — an abstract container holding the code being executed, its variables, and the 'this' value. There are three types:

1. Global Execution Context (GEC): Created once when the script starts. 'this' = global/window.
2. Function Execution Context (FEC): Created each time a function is called.
3. Eval Execution Context: Created for code in eval() — avoid in production.

Each context has two phases:

Creation Phase: Variable/function declarations are hoisted (var → undefined, functions → hoisted fully).

Execution Phase: Code runs line by line, variables assigned their values.

3.2 Call Stack

The Call Stack is a LIFO (Last In, First Out) data structure that tracks the execution of functions. When a function is called, its execution context is pushed onto the stack. When the function returns, it is popped off. If the stack grows too large (infinite recursion), you get a 'Maximum call stack size exceeded' error.

```
function third() { console.log('third'); }
function second() { third(); }
function first() { second(); }

first();

// Call Stack at deepest point:
// [ third() ] ← top
// [ second() ]
// [ first() ]
// [ main() ] ← bottom (global)

// Execution order: main → first → second → third
// Pop order: third → second → first → main
```

3.3 Heap Memory

The Heap is a large, unstructured region of memory where objects, arrays, and closures are stored. V8's garbage collector (Orinoco) manages heap memory using generational collection: new objects go to the 'Young Generation' (Scavenger GC, fast), long-lived objects are promoted to the 'Old Generation' (Mark-Sweep-Compact, slower).

```
// Stack: primitive values and references
let count = 42; // stored in stack
```

```
let name = 'Alice'; // stored in stack (reference to heap string)

// Heap: objects and arrays
let user = { id: 1, name: 'Bob' }; // object lives in heap
let arr = [1, 2, 3]; // array lives in heap

// When 'user' goes out of scope, GC eventually frees the heap memory
```

3.4 Callback Queue, Microtask Queue & Macrotask Queue

Node.js maintains multiple queues with different priorities. Understanding these is CRITICAL for interviews and debugging async code.

Queue	What Goes Here	Priority
process.nextTick queue	process.nextTick() callbacks	HIGHEST — runs before any other queue
Microtask queue	Promise .then/.catch, queueMicrotask()	2nd — after nextTick, before I/O
Macrotask / Callback queue	setTimeout, setInterval, setImmediate, I/O callbacks	LOWEST — after all microtasks

3.5 Event Loop — Full Deep Dive with Execution Order

Complete execution order rule:

1. Synchronous code runs first (call stack)
2. process.nextTick() queue is drained
3. Microtask queue (Promises) is drained
4. Event loop enters its phases (timers → poll → check → ...)
5. Between EACH event loop phase: nextTick + microtasks drain again

Classic Execution Order Example:

```
console.log('1 - sync');

setTimeout(() => console.log('2 - setTimeout'), 0);

Promise.resolve().then(() => console.log('3 - Promise'));

process.nextTick(() => console.log('4 - nextTick'));

setImmediate(() => console.log('5 - setImmediate'));

console.log('6 - sync end');

// OUTPUT:
// 1 - sync
// 6 - sync end
// 4 - nextTick ← nextTick drains first
```

```
// 3 - Promise ← microtask queue
// 2 - setTimeout ← timers phase
// 5 - setImmediate ← check phase
```

Async/Await Execution Order:

```
async function fetchData() {
console.log('B - inside async fn (sync part)');
const result = await Promise.resolve('data');
console.log('D - after await (microtask)');
return result;
}

console.log('A - before async call');
fetchData();
console.log('C - after async call (sync continues)');

// OUTPUT:
// A - before async call
// B - inside async fn (sync part)
// C - after async call (sync continues)
// D - after await (microtask) ← runs after sync stack is empty
```

Tricky: Nested Promises + nextTick

```
Promise.resolve().then(() => {
console.log('Promise 1');
process.nextTick(() => console.log('nextTick inside Promise'));
});

Promise.resolve().then(() => console.log('Promise 2'));

// OUTPUT:
// Promise 1
// nextTick inside Promise ← nextTick drains before next microtask
// Promise 2
```

3.6 setImmediate vs setTimeout(fn, 0)

Both defer execution, but they run in different event loop phases. When called at the top level, order is non-deterministic (depends on OS scheduling). Inside an I/O callback, setImmediate ALWAYS runs before setTimeout.

```
// Inside I/O callback — setImmediate wins:
const fs = require('fs');
fs.readFile(__filename, () => {
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
});
// OUTPUT (always):
```

```
// immediate
// timeout
```

3.7 Common Mistakes

■■ Common Mistakes

- *Blocking the event loop with heavy sync computation (for loops, JSON.parse on huge data)*
- *Assuming setTimeout(fn, 0) runs immediately — it is delayed to timer phase*
- *Forgetting that async/await does NOT make code parallel — it just pauses the current async function*
- *Not handling Promise rejections — causes UnhandledPromiseRejection crash in Node 15+*

Chapter 3 — Tricky Interview Questions

Q1. What is the difference between process.nextTick and setImmediate?

A: process.nextTick fires after the current operation completes, before the event loop continues — highest priority. setImmediate fires in the 'check' phase of the event loop — after I/O callbacks. nextTick can starve I/O if called recursively.

Q2. What runs first: Promise.then or setTimeout?

A: Promise.then (microtask) runs before setTimeout (macrotask). Microtasks drain completely before the event loop moves to the next phase.

Q3. What is the call stack overflow?

A: When too many function calls are pushed onto the call stack without returning (e.g., infinite recursion), V8 throws 'RangeError: Maximum call stack size exceeded'. Fix: use iteration or trampolining.

Q4. Explain async/await under the hood.

A: async functions return a Promise. await suspends the function at that point, saves its state, and yields control back to the event loop. When the awaited Promise resolves, the function resumes from the continuation as a microtask.

Q5. What is the difference between microtask and macrotask?

A: Microtasks: Promise callbacks, queueMicrotask, process.nextTick. They drain completely after each task before the next macrotask. Macrotasks: setTimeout, setInterval, setImmediate, I/O callbacks. One macrotask per event loop iteration.

CHAPTER SUMMARY

- ✓ *Execution Context: Global and Function — creation phase (hoisting) then execution phase*
- ✓ *Call Stack: LIFO — function pushed on call, popped on return*
- ✓ *Heap: unstructured object storage, managed by V8's generational GC*
- ✓ *Priority order: Sync → nextTick → Microtasks (Promises) → Macrotasks (setTimeout/I/O/setImmediate)*
- ✓ *async/await: syntactic sugar over Promises; resumes as a microtask after await resolves*
- ✓ *setImmediate vs setTimeout(0): setImmediate always wins inside I/O callbacks*

CHAPTER 4

Modules in Node.js

4.1 Why Modules?

Modules allow you to split your code into reusable, maintainable files. Without modules, all code would exist in one giant script — impossible to maintain at scale. Node.js supports two module systems: CommonJS (the original) and ES Modules (modern standard).

4.2 CommonJS (CJS)

DEFINITION

CommonJS is the original Node.js module system, introduced in 2009. It uses require() to import and module.exports (or exports) to export. Modules are loaded synchronously and cached after the first require().

Exporting (math.js):

```
// math.js
function add(a, b) { return a + b; }
function subtract(a, b) { return a - b; }
const PI = 3.14159;

// Option 1: Export object
module.exports = { add, subtract, PI };

// Option 2: Named exports shorthand
exports.multiply = (a, b) => a * b;
// WARNING: never do: exports = { ... }
// This breaks the reference to module.exports!
```

Importing (app.js):

```
// app.js
const math = require('./math'); // .js extension optional
const { add, PI } = require('./math'); // destructure

console.log(add(2, 3)); // 5
console.log(PI); // 3.14159

// require() is SYNCHRONOUS — file is read and executed immediately
// Cached: second require('./math') returns same object from cache
```

4.3 How require() Works Internally

INTERNAL WORKING

1. Resolve: Determine the file path (relative, absolute, or node_modules)

2. Load: Read the file from disk

3. Wrap: Node wraps the file in a function wrapper:

(function(exports, require, module, __filename, __dirname) { /* your code */ })

4. Execute: The wrapper function is called

5. Cache: module is stored in require.cache — subsequent requires return cached version

```
// Every CommonJS file is secretly wrapped in:
(function(exports, require, module, __filename, __dirname) {
// YOUR MODULE CODE HERE
// This is why __filename and __dirname work!
});
```

4.4 ES Modules (ESM)

ES Modules (import/export) are the official ECMAScript standard for modules, introduced in ES2015. Node.js supports them natively since v12 (stable in v14+). ESM modules are STATIC — imports are resolved at parse time, not runtime. They use .mjs extension OR 'type': 'module' in package.json.

Exporting (math.mjs):

```
// math.mjs (or package.json: { "type": "module" })

// Named exports
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }

// Default export
export default class Calculator {
add(a, b) { return a + b; }
}
```

Importing (app.mjs):

```
// app.mjs

// Named import
import { PI, add } from './math.mjs';

// Default import
import Calculator from './math.mjs';

// Import all
import * as Math from './math.mjs';

// Dynamic import (returns Promise)
const mod = await import('./math.mjs');
console.log(mod.add(2, 3)); // 5
```

4.5 CJS vs ESM — Comparison

Feature	CommonJS (CJS)	ES Modules (ESM)
Syntax	require() / module.exports	import / export
Loading	Synchronous	Asynchronous (static analysis)
Resolution	Runtime	Parse time (static)
Tree Shaking	Not possible	Supported by bundlers
Top-level await	Not supported	Supported
__dirname	Available	Not available (use import.meta.url)
Default in Node	Yes (.js files)	Requires .mjs or type:module
Interop	CJS can be require()'d in CJS	CJS can be imported in ESM; ESM cannot be require()'d
Use Case	Legacy, most npm packages	Modern code, Next.js, Deno

4.6 Common Mistakes

■■ Common Mistakes

- *exports = {...} reassigns the variable but NOT module.exports — exports nothing!*
- *Circular dependencies: A requires B which requires A — can cause undefined exports*
- *Mixing CJS and ESM carelessly — you cannot require() an ESM module*
- *Forgetting file extension in ESM imports (.mjs required for explicit resolution)*

Chapter 4 — Interview Questions

Q1. What is the difference between module.exports and exports?

A: exports is a reference to module.exports. Mutating exports (e.g., exports.fn = ...) works. Reassigning exports (exports = ...) breaks the reference and exports nothing. Always use module.exports = ... for replacing the entire export object.

Q2. How does Node.js cache modules?

A: After a module is first require()'d, Node stores it in require.cache keyed by its resolved file path. Subsequent requires return the cached module instantly without re-executing the file.

Q3. What is the module wrapper function?

A: Node wraps every CJS module in: (function(exports, require, module, __filename, __dirname) { }). This provides module-scoped variables and prevents global namespace pollution.

Q4. Can you use await at the top level in ESM?

A: Yes. Top-level await is supported in ES Modules. In CJS you must wrap await in an async function.

Q5. What is dynamic import()?

A: import() is an async function returning a Promise that resolves to the module's namespace object. It allows lazy loading of modules at runtime, works in both CJS and ESM contexts.

CHAPTER SUMMARY

✓ *CommonJS: require() / module.exports — synchronous, runtime resolution, default in Node.js*

✓ *ESM: import / export — static, async, tree-shakeable, top-level await supported*

✓ *require() wraps file in function — giving __filename, __dirname, module, exports*

✓ *Modules are cached after first load — singleton pattern by default*

✓ *Cannot require() an ESM module; use dynamic import() for CJS→ESM interop*

CHAPTER 5

Built-in Modules

Module	Category	Purpose
fs	File System	Read, write, delete, watch files and directories
path	Path Utilities	Manipulate file paths cross-platform
os	Operating System	System info: CPU, memory, hostname, platform
process	Process Info	Current process: args, env, exit, signals
events	Event Emitter	Pub/sub event system — base for many Node core classes
crypto	Cryptography	Hashing, HMAC, encryption, random bytes
http	HTTP Server/Client	Create HTTP servers and make HTTP requests
https	HTTPS Server/Client	Secure HTTP with TLS/SSL

5.1 fs (File System)

```
const fs = require('fs');
const fsP = require('fs/promises'); // Promise-based API

// --- ASYNC (callback) ---
fs.readFile('data.txt', 'utf8', (err, data) => {
if (err) throw err;
console.log(data);
});

// --- SYNC (blocking — avoid in servers!) ---
const data = fs.readFileSync('data.txt', 'utf8');

// --- PROMISE (recommended) ---
async function readAsync() {
const data = await fsP.readFile('data.txt', 'utf8');
console.log(data);
}

// Write file
await fsP.writeFile('output.txt', 'Hello Node.js!');

// Append to file
```

```
await fsP.appendFile('log.txt', 'Entry\n');

// Delete file
await fsP.unlink('temp.txt');

// Create directory
await fsP.mkdir('uploads', { recursive: true });

// Read directory
const files = await fsP.readdir('.');

// Check if file exists
try { await fsP.access('file.txt'); } catch { /* doesn't exist */ }

// Watch for changes
fs.watch('config.json', (event, filename) => {
console.log(`${filename} ${event}d`);
});

// Streams (for large files)
const readable = fs.createReadStream('huge.csv');
const writable = fs.createWriteStream('copy.csv');
readable.pipe(writable);
```

5.2 path

```
const path = require('path');

// Join paths (cross-platform)
path.join('/users', 'alice', 'docs', 'file.txt');
// → '/users/alice/docs/file.txt'

// Resolve to absolute path
path.resolve('src', 'app.js');
// → '/current/working/dir/src/app.js'

path.basename('/home/user/file.txt'); // 'file.txt'
path.dirname('/home/user/file.txt'); // '/home/user'
path.extname('report.pdf'); // '.pdf'

// Parse path into components
path.parse('/home/user/file.txt');
// { root:'/', dir:'/home/user', base:'file.txt', ext:'.txt', name:'file' }

// __dirname — directory of current file (CJS)
const fullPath = path.join(__dirname, 'templates', 'email.html');
```

```
// ESM equivalent
import { fileURLToPath } from 'url';
const __dirname = path.dirname(fileURLToPath(import.meta.url));
```

5.3 os

```
const os = require('os');

os.platform(); // 'linux', 'darwin', 'win32'
os.arch(); // 'x64', 'arm64'
os.hostname(); // machine hostname
os.homedir(); // '/home/alice'
os.tmpdir(); // '/tmp'
os.uptime(); // seconds since last boot
os.totalmem(); // total RAM in bytes
os.freemem(); // free RAM in bytes
os.cpus(); // array of CPU info objects
os.networkInterfaces(); // network interfaces info
os.EOL; // '\n' on Unix, '\r\n' on Windows
```

5.4 process

```
// process is a global — no require() needed

process.argv; // ['node', 'app.js', 'arg1', 'arg2']
process.env.NODE_ENV; // 'development' | 'production' | 'test'
process.env.PORT; // custom environment variable
process.cwd(); // current working directory
process.pid; // process ID
process.version; // Node.js version string
process.platform; // 'linux', 'darwin', 'win32'
process.memoryUsage(); // { rss, heapTotal, heapUsed, external }
process.hrtime.bigint(); // nanosecond-precision time

// Exit
process.exit(0); // 0 = success, non-zero = error
process.exit(1);

// Handle signals
process.on('SIGTERM', () => { /* graceful shutdown */ });
process.on('SIGINT', () => { /* Ctrl+C handler */ });

// Handle uncaught errors
process.on('uncaughtException', (err) => {
console.error('Fatal:', err);
process.exit(1);
});
```

```
process.on('unhandledRejection', (reason) => {
console.error('Unhandled Promise Rejection:', reason);
});

// stdin/stdout
process.stdout.write('Hello\n');
process.stdin.on('data', (chunk) => console.log('Got:', chunk.toString()));
```

5.5 crypto

```
const crypto = require('crypto');

// Hash (one-way)
crypto.createHash('sha256').update('password').digest('hex');

// HMAC (keyed hash)
crypto.createHmac('sha256', 'secret').update('data').digest('hex');

// Random bytes (tokens, salts)
const token = crypto.randomBytes(32).toString('hex');

// UUID v4
crypto.randomUUID(); // Node 14.17+

// Symmetric encryption (AES-256-GCM)
const key = crypto.randomBytes(32);
const iv = crypto.randomBytes(12);
const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
let encrypted = cipher.update('secret data', 'utf8', 'hex');
encrypted += cipher.final('hex');
const authTag = cipher.getAuthTag();

// Password hashing (async, uses thread pool)
crypto.pbkdf2('password', 'salt', 100000, 64, 'sha512', (err, key) => {
console.log(key.toString('hex'));
});

// Scrypt (modern, memory-hard)
const scryptKey = crypto.scryptSync('password', 'salt', 64);
```

5.6 http & https

```
const http = require('http');
const https = require('https');
const fs = require('fs');
```

```
// ■■■ Create HTTP Server ■■■
const server = http.createServer((req, res) => {
res.writeHead(200, { 'Content-Type': 'application/json' });
res.end(JSON.stringify({ message: 'Hello World' }));
});
server.listen(3000, () => console.log('Server on :3000'));

// ■■■ Make HTTP GET request ■■■
http.get('http://api.example.com/data', (res) => {
let body = '';
res.on('data', (chunk) => body += chunk);
res.on('end', () => console.log(JSON.parse(body)));
});

// ■■■ Create HTTPS Server ■■■
const options = {
key: fs.readFileSync('private.key'),
cert: fs.readFileSync('certificate.crt'),
};
https.createServer(options, (req, res) => {
res.end('Secure!');
}).listen(443);

// Note: In production use Express/Fastify on top of http module
```

5.7 events Module

```
const EventEmitter = require('events');

class MyEmitter extends EventEmitter {}
const myEmitter = new MyEmitter();

// Listen
myEmitter.on('greet', (name) => console.log(`Hello, ${name}!`));

// One-time listener
myEmitter.once('init', () => console.log('Initialized once'));

// Emit
myEmitter.emit('greet', 'Alice');
myEmitter.emit('init');
myEmitter.emit('init'); // this second call does nothing (once)

// Remove listener
const handler = () => {};
myEmitter.on('event', handler);
myEmitter.off('event', handler); // removes specific listener
```

```
// Max listeners warning (default 10)
myEmitter.setMaxListeners(20);

// Error event — always handle it!
myEmitter.on('error', (err) => console.error(err));
```

Chapter 5 — Interview Questions

Q1. What is the difference between fs.readFile and fs.readFileSync?

A: readFile is asynchronous — takes a callback, doesn't block. readFileSync is synchronous — blocks the event loop until complete. Never use Sync variants in server request handlers as they block all other requests.

Q2. Why should you avoid fs.existsSync to check before accessing a file?

A: It creates a TOCTOU (time-of-check/time-of-use) race condition. The file could be deleted between the check and the access. Use try/catch with fs.promises.access instead.

Q3. What is process.env and how do you use it?

A: process.env is an object containing environment variables. Use it for config: DB connection strings, API keys, PORT. Set via .env files (dotenv library) or OS environment. Never hardcode secrets in code.

Q4. What does crypto.randomBytes return and why use it over Math.random?

A: randomBytes returns cryptographically secure random data from the OS. Math.random is NOT cryptographically secure — predictable by attackers. Always use crypto.randomBytes for tokens, salts, session IDs.

Q5. What is the difference between http and https modules?

A: http is plain HTTP. https wraps http with TLS/SSL, requiring SSL certificates (key, cert). In production, use a reverse proxy (nginx) for HTTPS and run Node on plain HTTP internally.

CHAPTER SUMMARY

- ✓ *fs: async preferred (fs.promises / callback); never use Sync in server handlers*
- ✓ *path.join() for cross-platform paths; path.resolve() for absolute paths*
- ✓ *os: system information (CPU, RAM, platform) useful for monitoring and adaptive config*
- ✓ *process: global object — env vars, argv, signals, exit codes, memory usage*
- ✓ *crypto: sha256 hashing, randomBytes for tokens, pbkdf2/scrypt for passwords*
- ✓ *http: foundation for web frameworks; use createServer() for basic HTTP handling*

CHAPTER 6

NPM Ecosystem

6.1 What is npm?

npm (Node Package Manager) is the world's largest software registry with 2M+ packages. It ships with Node.js and provides: a CLI for installing packages, a registry (npmjs.com), and a standard for defining project metadata (package.json). Yarn and pnpm are alternative package managers compatible with the npm registry.

Common npm Commands:

Command	Purpose
npm init -y	Create package.json with defaults
npm install <pkg>	Install package and add to dependencies
npm install <pkg> --save-dev	Install and add to devDependencies
npm install	Install all deps from package.json
npm uninstall <pkg>	Remove package
npm update	Update all packages to latest compatible
npm run <script>	Run a script defined in package.json
npm publish	Publish package to npmjs.com registry
npm audit	Check for security vulnerabilities
npm ci	Clean install from package-lock.json (CI/CD)
npm list --depth=0	List top-level installed packages

6.2 package.json — Deep Dive

```
{
"name": "my-app",
"version": "1.0.0",
"description": "A sample Node.js application",
"main": "src/index.js",
"type": "module",
"scripts": {
"start": "node src/index.js",
"dev": "nodemon src/index.js",
"build": "tsc",
"test": "jest --coverage",
"lint": "eslint . --fix",
"prepare": "husky install"
},
"dependencies": {
```

```
"express": "^4.18.2",
"mongoose": "^7.0.0"
},
"devDependencies": {
"jest": "^29.0.0",
"nodemon": "^3.0.0",
"typescript": "^5.0.0"
},
"peerDependencies": {
"react": ">=18.0.0"
},
"engines": {
"node": ">=18.0.0"
},
"keywords": ["api", "rest", "nodejs"],
"author": "Your Name <you@example.com>",
"license": "MIT",
"private": true
}
```

6.3 package-lock.json

package-lock.json is auto-generated by npm. It records the EXACT version, integrity hash, and download URL of every installed package (including transitive dependencies). This ensures reproducible installs across machines and CI/CD. ALWAYS commit package-lock.json. Use npm ci in CI pipelines for deterministic installs.

package.json vs package-lock.json

package.json: human-written, declares version RANGES (^1.2.3)

package-lock.json: auto-generated, records EXACT version installed

npm install uses ranges → may install different versions on different machines

npm ci always uses package-lock.json → identical installs every time

6.4 Semantic Versioning (SemVer)

SemVer format: MAJOR.MINOR.PATCH (e.g., 4.18.2)

Type	When to increment	Example
MAJOR	Breaking API changes	3.0.0 → 4.0.0
MINOR	New backward-compatible features	4.17.0 → 4.18.0
PATCH	Backward-compatible bug fixes	4.18.1 → 4.18.2

Version Range Symbols:

Symbol	Meaning	Example	Allows
^	Compatible with version	^4.18.0	4.18.0 to <5.0.0 (MINOR+PATCH)
~	Approximately equivalent	~4.18.0	4.18.0 to <4.19.0 (PATCH only)
*	Any version	*	Latest (dangerous in production)
exact	Exact version	4.18.2	Only 4.18.2
>=	Greater than or equal	>=4.0.0	4.0.0 and above

6.5 Dependencies vs devDependencies vs peerDependencies

Type	Purpose	Installed in Production?	Example
dependencies	Runtime required	Yes	express, mongoose, lodash
devDependencies	Build/test only	No (npm install --production)	jest, eslint, typescript, nodemon
peerDependencies	Host package requirement	Not auto-installed	react (for a React component library)
optionalDependencies	Optional enhancement	Install attempted, failure ignored	Platform-specific bindings

6.6 npx

npx executes npm packages without globally installing them. It downloads the package temporarily, runs it, then removes it. Use it for: scaffolding tools, one-off scripts, running local project binaries.

```
# Run create-react-app without global install
npx create-react-app my-app

# Run a specific version
npx node@18 --version

# Run local project binary (preferred over global install)
npx jest
npx eslint src/

# Run remote script (use with caution!)
npx cowsay 'Hello Node.js'
```

Chapter 6 — Interview Questions

Q1. What is the difference between npm install and npm ci?

A: npm install installs from package.json ranges, updating package-lock.json if needed. npm ci installs EXACTLY from package-lock.json (fails if it's missing or out of sync). npm ci is faster and deterministic — use in CI/CD pipelines.

Q2. What is the ^ symbol in SemVer?

A: ^ (caret) allows compatible updates. ^4.18.2 installs any version >=4.18.2 and <5.0.0. It is the default prefix npm adds. ~ (tilde) is stricter — only PATCH updates.

Q3. What are peerDependencies?

A: peerDependencies declare packages that the host project must provide. Used for library/plugin packages that should not bundle their own copy. E.g., a React component library lists react as a peerDependency so consumers use their own React version.

Q4. Should you commit package-lock.json?

A: Yes, always commit package-lock.json for applications. It ensures identical dependency trees across developer machines, staging, and production. For publishable libraries, it's debatable — many omit it to let consumers resolve deps freely.

Q5. What is the difference between dependencies and devDependencies?

A: dependencies are required at runtime. devDependencies are only needed during development/testing. npm install --omit=dev skips devDependencies — used in production Docker images to reduce size.

CHAPTER SUMMARY

- ✓ *npm: world's largest registry, ships with Node.js; Yarn/pnpm are alternatives*
- ✓ *package.json: project metadata, scripts, and dependency version RANGES*
- ✓ *package-lock.json: EXACT versions — commit it; use npm ci in CI/CD*
- ✓ *SemVer: MAJOR.MINOR.PATCH — ^allows MINOR+PATCH, ~allows PATCH only*
- ✓ *dependencies: runtime; devDependencies: build/test; peerDependencies: host-provided*
- ✓ *npx: run packages without installing them globally*

CHAPTER 7

Asynchronous Programming

7.1 Why Asynchronous?

JavaScript is single-threaded. If we blocked for every I/O operation (file read, DB query, HTTP call), the entire application would freeze. Async programming allows initiating I/O without waiting, keeping the thread free to process other work. Node.js has evolved through three async paradigms.

7.2 Callbacks

DEFINITION

A callback is a function passed as an argument to another function, to be called when an async operation completes. Node.js uses the 'error-first callback' convention: the first argument is always an error (or null if none).

```
const fs = require('fs');

// Error-first callback pattern:
fs.readFile('users.json', 'utf8', function(err, data) {
if (err) {
console.error('Error reading file:', err.message);
return;
}
const users = JSON.parse(data);
console.log('Users:', users.length);
});
```

7.3 Callback Hell (Pyramid of Doom)

When async operations depend on each other, callbacks nest deeply — creating unmaintainable code known as 'callback hell' or the 'pyramid of doom'.

```
// Classic callback hell:
getUser(userId, function(err, user) {
if (err) return handleError(err);
getProfile(user.id, function(err, profile) {
if (err) return handleError(err);
getPosts(profile.id, function(err, posts) {
if (err) return handleError(err);
getComments(posts[0].id, function(err, comments) {
if (err) return handleError(err);
// 4 levels deep — hard to read and maintain!
console.log(comments);
});
```

```
});
});
});
```

7.4 Promises

DEFINITION

A Promise is an object representing the eventual completion or failure of an async operation. It is a placeholder for a value that will be available in the future. A Promise can be in one of three states: Pending, Fulfilled, or Rejected.

Promise States:

State	Meaning	Transition
Pending	Initial state — result not yet available	→ Fulfilled or Rejected
Fulfilled	Operation completed successfully — has a value	Terminal state
Rejected	Operation failed — has a reason (Error)	Terminal state

```
// Creating a Promise
function fetchUser(id) {
return new Promise((resolve, reject) => {
// Simulate async DB query
setTimeout(() => {
if (id > 0) {
resolve({ id, name: 'Alice' }); // Success
} else {
reject(new Error('Invalid user ID')); // Failure
}
}, 100);
});
}

// Consuming a Promise
fetchUser(1)
.then(user => {
console.log('User:', user.name);
return fetchProfile(user.id); // Chain!
})
.then(profile => console.log('Profile:', profile))
.catch(err => console.error('Error:', err.message))
.finally(() => console.log('Done'));

// Promise Combinators
// Run multiple promises in parallel:
const [users, products] = await Promise.all([fetchUsers(), fetchProducts()]);
```

```
// First to settle (resolve OR reject):
const result = await Promise.race([fetch('/a'), fetch('/b')]);

// All settle (never rejects):
const results = await Promise.allSettled([p1, p2, p3]);

// First to RESOLVE (ignores rejections until all reject):
const first = await Promise.any([p1, p2, p3]);
```

7.5 Internal Working of Promises

INTERNAL WORKING

When you call .then(), it returns a NEW Promise. The callback passed to .then() is scheduled as a microtask when the original Promise resolves. Microtasks run after the current synchronous code, before any macrotasks. This is why Promises always resolve asynchronously, even if resolve() is called synchronously.

```
const p = new Promise((resolve) => {
resolve(42); // resolve called SYNCHRONOUSLY
});

p.then(val => console.log('then:', val)); // still async!

console.log('sync code');

// Output:
// sync code
// then: 42
// The .then() callback runs as a microtask
// even though resolve() was called synchronously
```

7.6 Async/Await

DEFINITION

async/await is syntactic sugar over Promises introduced in ES2017. An async function always returns a Promise. The await keyword pauses execution of the async function until the Promise resolves, then returns the resolved value. This makes async code look and behave like synchronous code.

```
// Same logic as the callback hell example — much cleaner:
async function getUserData(userId) {
try {
const user = await getUser(userId);
const profile = await getProfile(user.id);
const posts = await getPosts(profile.id);
const comments = await getComments(posts[0].id);
return { user, profile, posts, comments };
} catch (err) {
console.error('Error in getUserData:', err.message);
```

```
throw err; // re-throw for caller to handle
}
}

// Sequential vs Parallel
// SLOW: sequential (each awaits before next starts)
const user1 = await fetchUser(1);
const user2 = await fetchUser(2);

// FAST: parallel (both start simultaneously)
const [u1, u2] = await Promise.all([fetchUser(1), fetchUser(2)]);

// async IIFE (top-level await alternative in CJS)
(async () => {
const data = await fetchData();
console.log(data);
})();
```

7.7 Error Handling in Async Code

```
// 1. try/catch with async/await
async function loadData() {
try {
const data = await riskyOperation();
return data;
} catch (err) {
// Handle specific error types
if (err instanceof NetworkError) { /* retry */ }
else throw err; // rethrow unknown errors
}
}

// 2. Helper: wrap async fn to avoid try/catch everywhere
async function to(promise) {
try {
const data = await promise;
return [null, data];
} catch (err) {
return [err, null];
}
}

const [err, user] = await to(fetchUser(1));
if (err) return handleError(err);
console.log(user);

// 3. Promise chain error handling
```

```
fetchUser(1)
.then(processUser)
.catch(err => {
if (err.code === 'NOT_FOUND') return defaultUser;
throw err; // rethrow if not handled
});
```

Chapter 7 — Interview Questions

Q1. What is callback hell and how do you avoid it?

A: Callback hell is deeply nested callbacks making code unreadable. Solve with: 1) Named functions (not anonymous), 2) Promises with .then() chaining, 3) async/await for synchronous-looking code.

Q2. What are the three states of a Promise?

A: Pending (initial), Fulfilled (resolved with value), Rejected (failed with error). Once fulfilled or rejected, a Promise is settled and cannot change state.

Q3. What is the difference between Promise.all and Promise.allSettled?

A: Promise.all rejects immediately if ANY promise rejects. Promise.allSettled waits for ALL promises and returns an array of {status, value/reason} — never rejects. Use allSettled when you want all results regardless of failures.

Q4. Why should you not forget to await a Promise?

A: Without await, you get a Promise object instead of the resolved value. The code continues before the async result is ready. Unhandled rejections can crash Node 15+. Always await or use .catch().

Q5. What is Promise chaining?

A: Each .then() returns a new Promise. Returning a value from .then() passes it to the next .then(). Returning a Promise from .then() chains the next handler to wait for THAT Promise to resolve.

CHAPTER SUMMARY

✓ Callbacks: error-first pattern (err, data). Good for simple async, but nests poorly.

✓ Promises: 3 states (pending/fulfilled/rejected). .then()/.catch()/.finally() chaining

✓ async/await: syntactic sugar over Promises — cleaner, linear code flow

✓ Use Promise.all for parallel ops; Promise.allSettled when all results needed regardless of errors

✓ Promises resolve as microtasks — after sync code, before macrotasks

✓ Always handle Promise rejections — unhandledRejection crashes Node.js 15+

CHAPTER 8

Error Handling

8.1 Types of Errors in Node.js

Error Type	Description	Example
Operational Errors	Expected failures at runtime — should be handled gracefully	Network timeout, file not found, invalid input, DB connection refused
Programmer Errors	Bugs in code — should crash and alert	TypeError: undefined.property, wrong argument type
System Errors	OS-level errors exposed as Error with .code	ENOENT (no file), ECONNREFUSED (refused), EADDRINUSE (port in use)

8.2 try/catch

```
// Synchronous code
try {
const json = JSON.parse('invalid json');
} catch (err) {
console.error('Parse error:', err.message);
}

// async/await
async function loadConfig() {
try {
const raw = await fs.promises.readFile('config.json', 'utf8');
const cfg = JSON.parse(raw);
return cfg;
} catch (err) {
if (err.code === 'ENOENT') {
console.warn('Config not found, using defaults');
return defaultConfig;
}
throw err; // rethrow unexpected errors
} finally {
console.log('loadConfig finished');
}
}
```

8.3 Custom Error Classes

```
// Always extend Error for custom errors:
class AppError extends Error {
constructor(message, statusCode = 500, code = 'INTERNAL_ERROR') {
```

```
super(message);
this.name = this.constructor.name;
this.status = statusCode;
this.code = code;
Error.captureStackTrace(this, this.constructor);
}
}

class NotFoundError extends AppError {
constructor(resource) {
super(`${resource} not found`, 404, 'NOT_FOUND');
}
}

class ValidationError extends AppError {
constructor(field, message) {
super(message, 400, 'VALIDATION_ERROR');
this.field = field;
}
}

// Usage
throw new NotFoundError('User');
// NotFoundError: User not found
// status: 404, code: 'NOT_FOUND'
```

8.4 Promise Error Handling

```
// WRONG: missing .catch — unhandled rejection!
fetchUser(1).then(user => processUser(user));

// CORRECT: always handle rejection
fetchUser(1)
.then(user => processUser(user))
.catch(err => {
if (err instanceof NotFoundError) {
return { default: true };
}
// Log and rethrow unknown errors
logger.error({ err }, 'Unexpected error in fetchUser');
throw err;
});

// async/await equivalent:
try {
const user = await fetchUser(1);
await processUser(user);
```

```
} catch (err) { /* same handling */ }
```

8.5 Global Error Handling

```
// 1. Uncaught synchronous exceptions
process.on('uncaughtException', (err, origin) => {
// Log the error
console.error('UNCAUGHT EXCEPTION:', err);
// Flush logs, close DB connections, then exit
process.exit(1); // Required — process state is unknown!
});

// 2. Unhandled Promise rejections
process.on('unhandledRejection', (reason, promise) => {
console.error('UNHANDLED REJECTION at:', promise, 'reason:', reason);
// In Node 15+, this crashes by default. Handle explicitly!
process.exit(1);
});

// 3. Express.js error middleware (4 args = error handler)
app.use((err, req, res, next) => {
const status = err.status || 500;
res.status(status).json({
error: {
message: err.message,
code: err.code || 'INTERNAL_ERROR',
}
});
});
```

8.6 Production Best Practices

- Never swallow errors with empty catch blocks — at minimum log them
- Always use process.exit(1) after uncaughtException — do not try to recover
- Use a process manager (PM2, systemd) to auto-restart crashed Node processes
- Differentiate operational errors (handle gracefully) from programmer errors (crash + alert)
- Add structured logging with error context: { err, userId, requestId }
- Use APM tools (Datadog, Sentry, New Relic) to track errors in production
- Always validate input at boundaries — fail early with clear ValidationError
- Set NODE_ENV=production — some libraries disable expensive dev checks

---

Chapter 8 — Interview Questions

Q1. What is the difference between operational and programmer errors?

A: Operational: expected runtime failures (network down, invalid input) — handle gracefully, return error response. Programmer: bugs (TypeError, undefined access) — should crash and alert developer, not try

to continue in unknown state.

Q2. Why should you always call process.exit(1) after uncaughtException?

A: After an uncaught exception, the process state is unknown — memory may be corrupted, connections may be in inconsistent state. Running further code risks data corruption. Exit immediately; let a process manager restart cleanly.

Q3. What happens if a Promise rejection is unhandled in Node 15+?

A: Node.js throws an UnhandledPromiseRejection and crashes the process. In older versions it only emitted a warning. Always add .catch() or try/catch around awaited Promises.

Q4. How do you create custom error classes?

A: Extend the built-in Error class. Call super(message) in the constructor, set this.name, custom properties (statusCode, code). Call Error.captureStackTrace to remove the constructor from the stack trace.

Q5. What is the Express error handling middleware signature?

A: 4 parameters: (err, req, res, next). Must have exactly 4 args for Express to recognise it as an error handler. Place it LAST after all routes and other middleware.

CHAPTER SUMMARY

- ✓ *Three error types: operational (handle), programmer (crash), system (code like ENOENT)*
- ✓ *try/catch: synchronous and async/await errors; .catch() for Promise chains*
- ✓ *Custom errors: extend Error class, add status/code properties, captureStackTrace*
- ✓ *Global handlers: uncaughtException + unhandledRejection — always exit(1) after*
- ✓ *Production: structured logging, APM, process managers, early validation*
- ✓ *Never swallow errors — always log or rethrow*

CHAPTER 9

Event Emitter

9.1 What is EventEmitter?

EventEmitter is a built-in Node.js class in the 'events' module that implements the Observer Pattern. It allows objects to emit named events and register listener functions that are called when those events occur. Many core Node.js classes (http.Server, fs.ReadStream, net.Socket) extend EventEmitter.

9.2 Internal Working

INTERNAL WORKING

EventEmitter internally maintains a Map (or object) from event names to arrays of listener functions. When emit() is called, it iterates the listener array synchronously, calling each function. This is synchronous — listeners run in the order they were registered, in the same call stack as emit().

```
// Simplified internal implementation:
class EventEmitter {
constructor() {
this._listeners = new Map(); // eventName → [fn, fn, fn]
}

on(event, fn) {
if (!this._listeners.has(event)) this._listeners.set(event, []);
this._listeners.get(event).push(fn);
return this; // enables chaining
}

emit(event, ...args) {
const listeners = this._listeners.get(event) || [];
listeners.forEach(fn => fn(...args)); // SYNC call
return listeners.length > 0;
}

off(event, fn) {
const list = this._listeners.get(event) || [];
this._listeners.set(event, list.filter(f => f !== fn));
}
}
```

9.3 Complete EventEmitter API

Method	Description
emitter.on(event, fn)	Add persistent listener

Method	Description
emitter.once(event, fn)	Add listener that fires only once
emitter.off(event, fn)	Remove specific listener (alias: removeListener)
emitter.removeAllListeners([event])	Remove all listeners for event (or all events)
emitter.emit(event, ...args)	Emit event synchronously with arguments
emitter.listeners(event)	Returns array of listeners for event
emitter.listenerCount(event)	Count of listeners for event
emitter.setMaxListeners(n)	Set max listeners before memory leak warning
emitter.prependListener(event, fn)	Add listener to beginning of listener array
EventEmitter.defaultMaxListeners	Default max listeners for ALL instances (default: 10)

9.4 Custom Events — Real-World Example

```
const EventEmitter = require('events');

class OrderService extends EventEmitter {
constructor(db, mailer) {
super();
this.db = db;
this.mailer = mailer;
}

async createOrder(orderData) {
const order = await this.db.orders.create(orderData);

// Emit events — decoupled from side effects!
this.emit('order:created', order);
return order;
}
}

const orderService = new OrderService(db, mailer);

// Separate concerns: listeners handle side effects
orderService.on('order:created', async (order) => {
await mailer.sendConfirmation(order.userEmail, order);
console.log('Confirmation email sent');
});

orderService.on('order:created', async (order) => {
await analytics.track('order_created', { orderId: order.id });
});

orderService.on('order:created', (order) => {
```

```
console.log(`[AUDIT] Order ${order.id} created at ${new Date()}`);
});

// Create an order — all 3 listeners fire
await orderService.createOrder({ product: 'Book', qty: 2 });
```

9.5 Error Events

```
// CRITICAL: Always attach an 'error' listener!
// Unhandled 'error' events CRASH the process with throw

const emitter = new EventEmitter();

// Without this, emitting 'error' throws and crashes:
emitter.on('error', (err) => {
console.error('Emitter error:', err.message);
});

emitter.emit('error', new Error('Something went wrong'));
// Handled: 'Emitter error: Something went wrong'
```

9.6 EventEmitter vs Promises — When to Use Each

Aspect	EventEmitter	Promise/async-await
Multiplicity	Multiple listeners per event	One resolution/rejection
Times fired	Fires multiple times over lifetime	Resolves/rejects ONCE
Use case	Streams, ongoing events, pub/sub	One-time async result
Error handling	Must handle 'error' event	.catch() / try-catch
Examples	http.Server, fs.ReadStream, Socket.io	fs.promises.readFile, fetch

---

Chapter 9 — Interview Questions

Q1. How does EventEmitter work internally?

A: It maintains a Map from event names to arrays of listener functions. emit() iterates the array synchronously, calling each listener. Listeners run in registration order in the same call stack.

Q2. What is the difference between on and once?

A: on registers a persistent listener called every time the event is emitted. once registers a listener that removes itself after being called the first time, firing only once.

Q3. What happens if you emit an 'error' event with no listeners?

A: Node.js throws the error (the first argument of emit), crashing the process. Always add an 'error' listener to EventEmitters to handle errors gracefully.

Q4. What is the memory leak warning about max listeners?

A: Node warns when more than 10 listeners are added to one event (default max). This prevents accidental listener accumulation from repeated on() calls without cleanup. Use setMaxListeners(n) if you

genuinely need more.

Q5. How is EventEmitter used in Node.js core?

A: http.Server extends EventEmitter (emits 'request', 'connection', 'close'). fs.ReadStream emits 'data', 'end', 'error'. net.Socket emits 'data', 'close', 'error'. This pattern runs throughout the Node.js ecosystem.

CHAPTER SUMMARY

✓ EventEmitter: Observer Pattern — emit events, register listeners with .on()/.once()

✓ emit() is SYNCHRONOUS — listeners run in order in same call stack

✓ Always handle 'error' events — unhandled 'error' emit crashes the process

✓ Many Node.js core objects (http.Server, ReadStream, Socket) extend EventEmitter

✓ Use EventEmitter for ongoing/multiple events; use Promises for one-time async results

✓ Watch for memory leaks — use .off() or .once() to clean up listeners

CHAPTER 10

Interview Questions Bank

50 Beginner Questions

Q1. What is Node.js?

A: Node.js is an open-source, cross-platform JavaScript runtime built on V8 that executes JS outside the browser, enabling server-side development.

Q2. Is Node.js a programming language?

A: No. Node.js is a runtime environment. JavaScript is the language. Node.js provides the environment to run JavaScript on the server.

Q3. What is the V8 engine?

A: V8 is Google's open-source C++ JS engine used in Chrome and Node.js. It JIT-compiles JS to machine code via Ignition (bytecode) + TurboFan (optimiser).

Q4. What is npm?

A: npm (Node Package Manager) is the default package manager for Node.js. It provides a CLI for installing packages and a registry (npmjs.com) with 2M+ packages.

Q5. What is package.json?

A: package.json is a JSON file describing a Node.js project: name, version, scripts, dependencies, devDependencies, and other metadata.

Q6. What is the difference between dependencies and devDependencies?

A: dependencies are needed at runtime; devDependencies are only for development/testing. npm install --omit=dev skips devDependencies.

Q7. What is the event loop?

A: The event loop is libuv's mechanism that continuously checks for pending tasks (I/O callbacks, timers, etc.) and executes them, enabling non-blocking async operations.

Q8. What is non-blocking I/O?

A: Non-blocking I/O means initiating an I/O operation without waiting for it to complete. The result is delivered via a callback when ready. The thread continues other work.

Q9. What is a callback?

A: A function passed as an argument to another function, to be called when an async operation completes. Node.js uses error-first callbacks: (err, data) => {}.

Q10. What is a Promise?

A: A Promise is an object representing the eventual result of an async operation. States: pending, fulfilled, rejected. Methods: .then(), .catch(), .finally().

Q11. What is async/await?

A: Syntactic sugar over Promises (ES2017). async declares a function returns a Promise. await pauses execution until the awaited Promise resolves.

Q12. What is the difference between synchronous and asynchronous code?

A: Synchronous code blocks — each line waits for the previous to complete. Async code initiates operations and continues — result arrives via callback/Promise.

Q13. What is CommonJS?

A: Node.js's original module system. Uses require() to import modules and module.exports to export. Loads synchronously.

Q14. What is the difference between require and import?

A: require() is CommonJS — runtime, synchronous, returns the module. import is ESM — static, parsed at compile time, async. Cannot mix in the same file.

Q15. What is module.exports?

A: The object that CommonJS exports when a file is require()'d. Setting module.exports = {} replaces the entire export; exports.fn = ... adds named properties.

Q16. What is __dirname?

A: __dirname is a CJS global variable containing the absolute directory path of the current module file.

Q17. What is process.env?

A: An object containing the process's environment variables. Used for config, secrets, feature flags. Set via OS, .env files (dotenv), or CI/CD systems.

Q18. How do you read a file in Node.js?

A: Use fs.promises.readFile(path, encoding) for async, fs.readFileSync() for sync (avoid in servers). The fs module provides full file system access.

Q19. What is the http module?

A: A built-in Node.js module for creating HTTP servers and making HTTP requests. Foundation for web frameworks like Express.

Q20. What is Express.js?

A: A minimal, flexible Node.js web framework providing routing, middleware support, and HTTP utilities. Most popular Node.js web framework.

Q21. What is middleware?

A: Functions in an Express app that have access to (req, res, next). They process requests in sequence — authentication, logging, parsing, error handling.

Q22. What is the path module?

A: A built-in module for manipulating file paths. path.join(), path.resolve(), path.basename(), path.extname() — handles cross-platform separators.

Q23. What is the os module?

A: Built-in module for OS information: platform, architecture, CPUs, memory, hostname, network interfaces, home directory, temp directory.

Q24. What is an EventEmitter?

A: A class from the 'events' module implementing the Observer Pattern. emit() triggers events; on() registers listeners; once() registers one-time listeners.

Q25. What is callback hell?

A: Deeply nested callbacks making code unreadable and hard to maintain. Solutions: named functions, Promises, async/await.

Q26. What does process.exit() do?

A: Terminates the current Node.js process. process.exit(0) = success, process.exit(1) = error. Triggers 'exit' event before terminating.

Q27. What is SemVer?

A: Semantic Versioning: MAJOR.MINOR.PATCH. MAJOR = breaking changes, MINOR = new features (backward-compatible), PATCH = bug fixes.

Q28. What is the ^ symbol in package.json?

A: Caret (^) allows compatible updates: ^1.2.3 accepts >=1.2.3 and <2.0.0. It allows MINOR and PATCH updates but not MAJOR.

Q29. What is npx?

A: A tool (ships with npm) to run npm packages without global installation. Downloads, executes, then discards the package.

Q30. What is the difference between == and === in JavaScript?

A: == compares with type coercion; === compares without (strict). Always use === in Node.js to avoid unexpected coercion bugs.

Q31. What is a Buffer in Node.js?

A: A Buffer is a fixed-size raw binary data container (like a Uint8Array). Used for I/O operations: network packets, file reads, streams.

Q32. What is a stream?

A: A stream is an abstract interface for reading or writing data in chunks. Types: Readable, Writable, Duplex, Transform. Used for large files, HTTP, compression.

Q33. What is the crypto module?

A: Built-in module for cryptographic operations: hashing (SHA-256), HMAC, encryption (AES), random bytes, UUID, password hashing (pbkdf2, scrypt).

Q34. What is REST API?

A: Representational State Transfer — architectural style for APIs using HTTP methods (GET, POST, PUT, DELETE) and stateless requests.

Q35. What is JSON.stringify vs JSON.parse?

A: JSON.stringify converts JS object to JSON string. JSON.parse converts JSON string to JS object. Both throw on invalid input.

Q36. What is the global object in Node.js?

A: 'global' (or globalThis) is Node's global object — equivalent to 'window' in browsers. Variables without var/let/const attach to it.

Q37. What is the difference between let, const, and var?

A: var: function-scoped, hoisted. let: block-scoped, not hoisted (TDZ). const: block-scoped, must be initialised, can't be reassigned (but object properties can change).

Q38. What is hoisting?

A: JavaScript moves var declarations and function declarations to the top of their scope during compilation. let/const are hoisted but NOT initialised (Temporal Dead Zone).

Q39. What is a closure?

A: A closure is a function that retains access to variables in its outer scope even after the outer function has returned. Enables data encapsulation, memoisation, and callbacks.

Q40. What is the spread operator?

A: ... in arrays/objects copies elements. [...arr] clones an array. {...obj} clones an object. In function calls, spreads array as arguments.

Q41. What is destructuring?

A: Extracting values from arrays/objects into variables: const { name, age } = user; const [first, second] = arr; Supports defaults and renaming.

Q42. What is a template literal?

A: Backtick strings (``) supporting embedded expressions ${expr} and multiline strings. Cleaner than string concatenation.

Q43. What is the difference between null and undefined?

A: undefined: variable declared but not assigned. null: intentionally empty value. typeof null === 'object' is a historic JS bug.

Q44. What is the purpose of package-lock.json?

A: Records the EXACT version and integrity of every installed package for reproducible installs. Always commit it; use npm ci in CI/CD.

Q45. What is REPL in Node.js?

A: Read-Eval-Print-Loop: interactive shell for running JS code. Start with 'node' command. Useful for quick experimentation.

Q46. What is process.argv?

A: An array of command-line arguments. [0] = 'node', [1] = script path, [2+] = arguments passed to the script.

Q47. What is the setTimeout function?

A: Schedules a callback to run after at least 'delay' ms. Not a Node.js-specific feature — inherited from browser JS but implemented via libuv timers.

Q48. What is the difference between setInterval and setTimeout?

A: setTimeout fires once after delay. setInterval fires repeatedly at interval. clearTimeout/clearInterval cancels them.

Q49. What is JSON?

A: JavaScript Object Notation — a lightweight data interchange format. Supports strings, numbers, booleans, null, arrays, and objects. Human-readable, language-agnostic.

Q50. How do you handle environment variables?

A: Set in OS, .env file (dotenv library), or CI/CD. Access via process.env.VAR_NAME. Never hardcode secrets — use environment variables.

50 Intermediate Questions

Q1. Explain the Node.js event loop phases in detail.

A: 6 phases: 1) timers (setTimeout/setInterval), 2) pending I/O callbacks, 3) idle/prepare (internal), 4) poll (new I/O), 5) check (setImmediate), 6) close callbacks. Between each phase, nextTick and microtasks drain.

Q2. What is the difference between process.nextTick and Promise.then?

A: Both are microtasks. nextTick queue drains BEFORE the Promise microtask queue. nextTick can starve Promise callbacks if called recursively.

Q3. What happens when you call require() on a module twice?

A: Node.js caches modules after first load. Second require() returns the cached exports object — the module code does NOT re-execute.

Q4. Explain the module wrapper function.

A: Node wraps CJS files in: (function(exports, require, module, __filename, __dirname) { your code }). This provides module-scope variables and isolates the module from the global scope.

Q5. What is a Worker Thread and when should you use it?

A: Worker Threads (worker_threads module) enable true parallel JS execution in Node.js. Use for CPU-intensive tasks (image processing, compression, ML) to avoid blocking the event loop.

Q6. What is cluster module?

A: Creates child processes (workers) sharing server ports. Master process forks workers = one per CPU core. Allows Node.js to utilise multi-core CPUs. PM2 provides this out of the box.

Q7. What is the difference between fork() and spawn() in child_process?

A: spawn() launches a command in a new process with streaming I/O. fork() is spawn() specialised for Node.js scripts, establishing IPC channel for message passing between parent and child.

Q8. How do streams improve memory efficiency?

A: Streams process data in chunks (default 16KB). Reading a 1GB file with fs.readFile loads it all into RAM. Using ReadStream processes one chunk at a time — constant memory usage regardless of file size.

Q9. What is backpressure in streams?

A: Backpressure occurs when a writable stream cannot consume data as fast as the readable produces it. The write() method returns false; the readable should pause until 'drain' event fires. pipe() handles this automatically.

Q10. Explain Promise.all vs Promise.race vs Promise.allSettled vs Promise.any.

A: all: resolves when ALL resolve (rejects if any rejects). race: resolves/rejects as soon as the FIRST settles. allSettled: resolves when ALL settle, returns status+value for each. any: resolves when the FIRST resolves (rejects only if ALL reject).

Q11. What is the significance of async/await error handling at the top level?

A: In CJS, top-level await is not supported — use async IIFE. In ESM, top-level await is available. Unhandled top-level rejections crash the process in Node 15+ (unhandledRejection).

Q12. How does Node.js handle circular dependencies in require()?

A: Node returns an incomplete (partially-filled) module.exports at the point of the circular dependency. This can cause undefined imports. Refactor to avoid cycles by extracting shared logic to a third module.

Q13. What is the difference between .env and environment variables?

A: .env files are local developer convenience (via dotenv). True environment variables come from the OS / deployment platform. In production, use platform secrets management (AWS Secrets Manager, Vault, K8s Secrets) — never commit .env.

Q14. What is HTTP keep-alive?

A: HTTP persistent connection that reuses TCP connection for multiple requests. Avoids TCP handshake overhead. Node.js HTTP Agent enables it by default. Important for high-throughput APIs.

Q15. What is the role of the thread pool in libuv?

A: libuv's thread pool (default 4 threads, max 128) handles tasks the OS cannot make truly async: file I/O, DNS lookup, crypto, zlib. These run on background threads while the event loop continues.

Q16. What is a memory leak in Node.js and how do you detect it?

A: Memory leak: objects retain references and are never GC'd — heap grows indefinitely. Detect with: process.memoryUsage(), heapdump, node --inspect + Chrome DevTools memory profiler, clinic.js.

Q17. What is the difference between HTTP/1.1 and HTTP/2 in Node.js?

A: HTTP/1.1: one request per connection (or keep-alive). HTTP/2: multiplexing (multiple requests over one connection), header compression, server push. Node.js supports HTTP/2 via http2 module.

Q18. What is CORS and how do you handle it in Node.js?

A: Cross-Origin Resource Sharing: browser security policy blocking requests to different origins. Handle with cors npm package (Express) or manually setting Access-Control-Allow-Origin headers.

Q19. Explain JWT authentication flow.

A: 1) User logs in → server creates JWT (header.payload.signature). 2) Client stores JWT. 3) Client sends JWT in Authorization: Bearer header. 4) Server verifies signature with secret — no DB lookup needed.

Q20. What is rate limiting and how do you implement it?

A: Rate limiting restricts how many requests a client can make in a time window. Implement with: express-rate-limit (in-memory), Redis-backed rate limiter for distributed systems (rate-limiter-flexible).

Q21. What is the difference between authentication and authorisation?

A: Authentication: who are you? (login, JWT, OAuth). Authorisation: what are you allowed to do? (RBAC, permissions). Always authenticate before authorising.

Q22. What is HTTPS and how do you set it up in Node.js?

A: HTTPS = HTTP + TLS. Requires SSL certificate (private key + certificate). Use https.createServer({key, cert}, handler). In production, terminate TLS at nginx/load balancer, run Node on plain HTTP internally.

Q23. What is graceful shutdown?

A: Stopping the server without dropping in-flight requests. 1) Stop accepting new connections (server.close()). 2) Finish processing existing requests. 3) Close DB connections. 4) Exit. Listen for SIGTERM for container/PM2 shutdowns.

Q24. What is the child_process module used for?

A: Spawning external processes: shell commands, Python scripts, FFmpeg, etc. Methods: exec() (buffered), spawn() (streaming), execFile() (direct file), fork() (Node.js child with IPC).

Q25. What is a RESTful API design principle?

A: Stateless, client-server, cacheable, layered system, uniform interface (HTTP verbs + resource URLs). Each request contains all info needed — no server-side session state.

Q26. How do you test Node.js applications?

A: Unit tests: Jest/Mocha/Vitest for individual functions. Integration tests: Supertest for HTTP endpoints. E2E: Playwright/Cypress. Use test doubles (mocks, stubs, spies) to isolate dependencies.

Q27. What is dependency injection?

A: Passing dependencies as arguments instead of hardcoding them. Enables testability (inject mocks), flexibility (swap implementations), and loose coupling between modules.

Q28. What is the difference between SQL and NoSQL databases?

A: SQL (PostgreSQL, MySQL): structured, relations, ACID, joins. NoSQL (MongoDB, Redis): flexible schema, horizontal scaling, eventual consistency. Choose based on data structure and query patterns.

Q29. What is connection pooling?

A: Maintaining a pool of pre-established DB connections. Reused for requests instead of creating new connections each time. Critical for performance — creating a DB connection takes 20-100ms.

Q30. What is the crypto.timingSafeEqual function?

A: Compares two Buffers in constant time, preventing timing attacks. Use when comparing secrets (tokens, MACs) — regular === leaks timing info about where strings differ.

Q31. What is the Node.js inspector?

A: Built-in debugger interface. Run with node --inspect app.js. Connect via Chrome DevTools (chrome://inspect), VS Code debugger, or WebSocket client for breakpoints, profiling, heap snapshots.

Q32. What is PM2?

A: Production process manager for Node.js. Features: auto-restart on crash, clustering (multi-core), log management, 0-downtime reloads, startup scripts, monitoring dashboard.

Q33. What is docker and how does it relate to Node.js?

A: Docker packages Node.js apps into containers — isolated environments with their own Node version, npm packages, and config. Ensures identical environments from dev to production.

Q34. What is a microservices architecture?

A: Application divided into small, independent services communicating via HTTP/gRPC/message queues. Each service is independently deployable. Node.js is popular for microservices due to low footprint and async I/O.

Q35. What is GraphQL?

A: API query language letting clients request exactly the data they need (no over/under-fetching). Schema-first design, strongly typed. Node.js servers: Apollo Server, Yoga.

Q36. What is WebSocket?

A: Full-duplex communication protocol over a single TCP connection. After HTTP upgrade handshake, both client and server can send messages at any time. Use for real-time features. Node.js: ws library, Socket.io.

Q37. What is server-sent events (SSE)?

A: One-directional real-time updates from server to client over HTTP. Simpler than WebSocket for push notifications, live feeds, progress updates. Uses text/event-stream content type.

Q38. What is the difference between horizontal and vertical scaling?

A: Vertical: bigger server (more CPU/RAM). Horizontal: more servers (Node cluster, load balancer). Node.js excels at horizontal scaling — stateless services behind a load balancer with shared Redis/DB.

Q39. What is Redis and how is it used with Node.js?

A: Redis is an in-memory key-value store. Used for: session storage, caching, rate limiting, pub/sub messaging, queues. Node.js: ioredis or node-redis client.

Q40. What is message queueing?

A: Async communication pattern: producer sends messages to a queue; consumers process them independently. Decouples services, handles spikes. Tools: RabbitMQ, AWS SQS, Kafka, BullMQ (Redis-based).

Q41. What is the Streams pipe() method?

A: readable.pipe(writable) automatically moves data from readable to writable, handling backpressure. Chains: readable.pipe(transform).pipe(writable). Returns the destination for chaining.

Q42. What is the net module?

A: Provides TCP/IPC socket API for building low-level network applications. Underlies the http module. Use for: TCP servers, IPC communication, custom protocols.

Q43. What is async_hooks?

A: Node.js API for tracking async resources (their creation and destruction). Used for distributed tracing (propagating request IDs across async boundaries), debugging, APM tools.

Q44. What is the difference between JSON and BSON?

A: JSON: text-based, human-readable, string keys/values. BSON (Binary JSON): binary format used by MongoDB, supports more types (Date, ObjectId, Binary), more compact, faster to parse.

Q45. What is input validation and why is it critical?

A: Validating and sanitising user input at API boundaries. Prevents injection attacks (SQL injection, NoSQL injection, XSS), bad data in DB. Libraries: Joi, Zod, express-validator.

Q46. What is the difference between unit and integration tests?

A: Unit tests: test individual functions/modules in isolation using mocks. Integration tests: test multiple components working together (e.g., route + service + DB). Both are necessary.

Q47. What is the difference between EventEmitter and RxJS Observables?

A: EventEmitter is simple pub/sub — no lazy evaluation, no operators. RxJS Observables are lazy, composable streams with 100+ operators (map, filter, merge, debounce). Overkill for simple cases;

powerful for complex async flows.

Q48. What is caching and what are its strategies?

A: Caching: storing computed/fetched data for fast future access. Strategies: Cache-Aside (app checks cache first), Write-Through (write to cache+DB), Write-Back (write to cache, async to DB). TTL prevents stale data.

Q49. What is the significance of NODE_ENV?

A: NODE_ENV=production enables optimisations in Express (disables verbose errors), libraries, and allows conditional code. Set it in deployment. Never use NODE_ENV=development in production.

Q50. What is ACID in databases?

A: Atomicity (all-or-nothing), Consistency (valid state), Isolation (concurrent transactions don't interfere), Durability (committed data persists). SQL databases guarantee ACID; most NoSQL trade it for performance/scale.

CHAPTER 11

Revision Notes, Cheat Sheets & Common Traps

11.1 Core Concepts Cheat Sheet

Concept	One-Line Summary
Node.js	JS runtime on V8 + libuv for async server-side code
V8	Google's JS engine: Ignition bytecode + TurboFan JIT compiler
libuv	C library: event loop + thread pool + async I/O across platforms
Event Loop	Continuously runs phases: timers→poll→check. Microtasks drain between phases
Non-blocking I/O	Initiate I/O, register callback, continue. Callback fires when OS signals completion
Call Stack	LIFO execution context tracker. Full stack = RangeError
Microtask Queue	Promises + nextTick. Drained between EVERY macrotask
Macrotask Queue	setTimeout, setInterval, setImmediate, I/O callbacks. One per loop iteration
CommonJS	require() / module.exports — synchronous, cached, wrapped in module function
ES Modules	import/export — static, async, tree-shakeable, top-level await
npm	Package manager + registry. package.json = ranges; package-lock.json = exact versions
SemVer	MAJOR.MINOR.PATCH. ^=MINOR+PATCH allowed; ~=PATCH only
Callback	Error-first fn passed to async op: (err, data) => {}
Promise	Pending → Fulfilled/Rejected. .then()/.catch()/.finally()
async/await	Syntactic sugar over Promises. await pauses async fn, resumes as microtask
EventEmitter	Observer Pattern. emit() triggers event; on() adds listener; emit is SYNC
Stream	Process data in chunks. Readable/Writable/Duplex/Transform. pipe() handles backpressure
Buffer	Raw binary data. Fixed size. Used for I/O, crypto, file ops
process	Global: env, argv, exit, signals, stdin/stdout, memoryUsage

11.2 Async Execution Order Quick Reference

```
// EXECUTION ORDER (memorise this!):
// 1. Synchronous code (call stack)
// 2. process.nextTick() queue (all of it)
// 3. Promise microtask queue (all of it)
// 4. setTimeout (timers phase)
// 5. setImmediate (check phase) — after I/O: ALWAYS before setTimeout
// Between each event loop phase: nextTick + microtasks drain again

console.log('A'); // 1st
setTimeout(() => console.log('B'), 0); // 4th (timers)
Promise.resolve().then(() => console.log('C')); // 3rd (microtask)
process.nextTick(() => console.log('D')); // 2nd (nextTick)
setImmediate(() => console.log('E')); // 5th (check)
console.log('F'); // 1st (sync)

// OUTPUT: A F D C B E
```

11.3 Module System Quick Reference

Action	CommonJS	ES Modules
Import module	const x = require('./mod')	import x from './mod.js'
Import named	const {a,b} = require('./mod')	import { a, b } from './mod.js'
Import all	const mod = require('./mod')	import * as mod from './mod.js'
Export default	module.exports = value	export default value
Export named	exports.fn = () => {}	export function fn() {}
Dynamic import	require() (sync)	await import('./mod.js') (async)
File extension	.js or none	.mjs or .js with type:module
__dirname	Available globally in wrapper	import.meta.url + fileURLToPath
Top-level await	Not supported	Supported

11.4 Common Traps & Gotchas

■■ COMMON TRAPS

TRAP: exports = {} does NOT work

FIX: exports is a reference to module.exports. Reassigning exports breaks the reference. Use module.exports = {} to replace the export object.

TRAP: async forEach doesn't await

FIX: arr.forEach(async fn) fires all callbacks without awaiting. Use for...of with await, or Promise.all(arr.map(async fn)).

TRAP: JSON.parse throws on invalid JSON

FIX: Always wrap JSON.parse in try/catch. One malformed byte crashes your handler.

TRAP: require() is cached — mutating exports affects all importers
FIX: Since modules are singletons, mutating a required object affects every file that required it. Return new objects from factory functions if isolation is needed.

TRAP: setTimeout(fn, 0) is NOT immediate
FIX: It waits until the timers phase of the next event loop iteration. process.nextTick or Promise.resolve().then() run sooner.

TRAP: Uncaught promise rejections crash Node 15+
FIX: Always handle Promise rejections. Add process.on('unhandledRejection') as a safety net, but fix the root cause.

TRAP: fs.readFileSync blocks the event loop
FIX: Sync methods block ALL other requests while they run. Never use *Sync methods in server request handlers.

TRAP: Forget to close DB connections on exit
FIX: Add process.on('SIGTERM', gracefulShutdown) to close DB pools, flush caches, finish in-flight requests before exit.

TRAP: Using Math.random() for security
FIX: Math.random() is NOT cryptographically secure. Use crypto.randomBytes() for tokens, salts, session IDs.

TRAP: Not setting NODE_ENV=production
FIX: Many libraries (Express) have development-mode overhead (verbose errors, etc.) disabled only when NODE_ENV=production. Always set it in deployment.

11.5 Best Practices Quick Reference

Area	Best Practice
Error Handling	Always handle errors. Never empty catch. Use custom error classes. Global handlers + exit(1).
Async	Prefer async/await. Use Promise.all for parallel. Handle all rejections.
Security	Use crypto.randomBytes not Math.random. Validate all input. Use helmet, rate limiting, CORS.
Config	Use process.env for all config. dotenv for dev. Secrets manager for prod. Never commit .env.
Performance	Avoid *Sync in handlers. Use streams for large data. Set UV_THREADPOOL_SIZE. Profile with clinic.js.
Modules	Prefer ESM for new projects. Always commit package-lock.json. Use npm ci in CI/CD.
Testing	Unit + integration tests. Use jest/vitest. Mock external dependencies. Aim for >80% coverage.

Area	Best Practice
Logging	Use structured logging (pino, winston). Include requestId. Log errors with stack traces.
Shutdown	Handle SIGTERM for graceful shutdown. Close DB + stop server before exit.
Production	Set NODE_ENV=production. Use PM2/systemd. Monitor with APM. Use Docker for consistency.

11.6 Built-in Modules Quick Reference

Module	Key APIs
fs / fs/promises	readFile, writeFile, appendFile, unlink, mkdir, readdir, watch, createReadStream
path	join, resolve, basename, dirname, extname, parse, sep, posix
os	platform, arch, hostname, homedir, tmpdir, cpus, totalmem, freemem, uptime, EOL
process	env, argv, cwd, pid, exit, on(SIGTERM), stdout, stdin, memoryUsage, hrtime
events	EventEmitter: on, once, off, emit, listeners, listenerCount, setMaxListeners
crypto	createHash, createHmac, randomBytes, randomUUID, pbkdf2, scrypt, createCipheriv
http/https	createServer, get, request, IncomingMessage, ServerResponse
child_process	exec, execFile, spawn, fork
worker_threads	Worker, parentPort, workerData, isMainThread
cluster	fork, isMaster, isWorker, worker, workers
stream	Readable, Writable, Duplex, Transform, pipeline, finished
url	URL, URLSearchParams, fileURLToPath, pathToFileURL
util	promisify, callbackify, inspect, format, inherits
net	createServer, createConnection, Socket
dns	lookup, resolve, resolve4, resolve6, reverse
zlib	gzip, gunzip, deflate, inflate, createGzip, createGunzip

11.7 Frequently Asked Questions

Q1. Is Node.js single-threaded?

A: The JavaScript event loop is single-threaded. But libuv uses a thread pool for file I/O, DNS, crypto. Worker Threads add true multi-threading for CPU-heavy JS code.

Q2. Can Node.js handle 10,000 concurrent connections?

A: Yes. Non-blocking I/O means the main thread is never idle waiting. Each connection registers an event; callbacks fire as I/O completes. Netflix, LinkedIn, NASA use Node for high concurrency.

Q3. Should I use TypeScript with Node.js?

A: Highly recommended for large projects. TypeScript adds type safety, better IDE support, refactoring confidence. Use ts-node for development, compile to JS for production.

Q4. What is the best framework for Node.js?

A: Express: most popular, minimal, huge ecosystem. Fastify: fastest, schema validation built-in. NestJS: opinionated, TypeScript-first, Angular-inspired (great for enterprise). Hono: ultra-fast, edge-compatible.

Q5. How do I scale Node.js?

A: Horizontally: multiple Node instances behind load balancer (nginx/HAProxy). Use cluster module or PM2 for multi-core. Share state via Redis (sessions, caches, queues). Stateless design is key.

Q6. What ORM should I use?

A: Prisma: type-safe, auto-generated client, great DX, supports PostgreSQL/MySQL/SQLite/MongoDB. Sequelize: mature, feature-rich. TypeORM: TypeScript-first, decorator-based. Mongoose: MongoDB ODM.

Q7. How do I secure a Node.js API?

A: HTTPS always. Helmet (security headers). CORS policy. Rate limiting. Input validation (Zod/Joi). JWT authentication. Environment variables for secrets. Regular npm audit. OWASP Top 10 awareness.

Q8. When should I use Worker Threads?

A: When you have CPU-intensive JS work: image processing, PDF generation, large JSON transformations, ML inference. Don't use for I/O — the event loop already handles that efficiently.

End of Part 1

Node.js Complete Learning & Interview Handbook — Part 1: Fundamentals to Intermediate

Coming in Part 2: Express.js Deep Dive · REST API Design · Authentication & JWT · Databases (MongoDB + PostgreSQL) · Testing · Docker · Microservices · Performance & Monitoring